

Parallel Programming Assignment Report

Problem Information

Problem Title: MPI Point-to-Point Communication

Lecture Number: chapter

Problem Description

The objective is to implement a fundamental parallel program using the Message Passing Interface (MPI). The goal is to demonstrate basic point-to-point communication between two distinct processes: one process (the sender) will transmit a "Hello World" message, and another process (the receiver) will capture and display it

Parallelization Approach

MPI allows communication between processes that do not share memory. The approach uses the process rank to assign different tasks to different processes (SPMD model), where process 0 is the sender and process 1 is the receiver.

Solution Explanation

MPI initializes the environment, assigns ranks to processes, then uses MPI_Send and MPI_Recv for point-to-point message passing between processes.

Finally, MPI_Finalize terminates the MPI execution environment.

Source Code

```
#include <mpi.h>
#include <iostream>
#include <string>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    if (rank == 0) {
        std::string message = "Hello World";
        MPI_Send(message.c_str(), message.size() + 1, MPI_CHAR, 1, 0, MPI_COMM_WORLD);
    }
    else if (rank == 1) {
        char buffer[100];
        MPI_Recv(buffer, 100, MPI_CHAR, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        std::cout << "Process 1 received: " << buffer << std::endl;
    }
    MPI_Finalize();
    return 0;
}
```

Execution Results

```
C:\Users\DELL\Desktop\FCI4.2\network_programing\Apps\projectNP\shahd\parallel\x64\Debug>mpirun -n 2 parallel.exe
Process 1 received: Hello World
C:\Users\DELL\Desktop\FCI4.2\network_programing\Apps\projectNP\shahd\parallel\x64\Debug>
```

Conclusion

MPI programs must explicitly manage the transfer of information between separate process memory spaces.